8

调度：多级反馈队列

本章要设计的，是调度领域最著名的方法之一：**多级反馈队列**（Multi-level Feedback Queue，MLFQ）。多级反馈队列（MLFQ）调度程序（scheduler）最早由 Corbato 等人于 1962 年 [C+62] 在一个叫兼容分时系统（Compatible Time-Sharing System，CTSS）的系统中提出；这项工作，连同后来在 Multics 上的工作，让 ACM 把它的最高荣誉**图灵奖**（Turing Award）授予了 Corbato。此后多年，这种调度程序不断改进，演变成今天一些现代系统里的实现。

MLFQ 要解决的根本问题有两方面。首先，它希望优化周转时间（turnaround time）。上一章讲过，这要靠先运行较短的作业来实现；可惜操作系统一般并不知道一个作业要运行多久，而 SJF（或 STCF）这类算法恰恰需要这种信息。其次，MLFQ 希望让系统对交互式用户（即坐在那里盯着屏幕、等进程（process）完成的用户）显得反应灵敏，从而把响应时间（response time）降到最低；遗憾的是，像时间片轮转（Round-Robin）这样的算法虽然降低了响应时间，周转时间却很糟糕。于是问题来了：既然我们通常对进程一无所知，怎样才能构建一个达成这些目标的调度程序？调度程序如何在系统运行时了解它所运行的作业有什么特点，从而做出更好的调度决策？

关键问题：
没有完备的信息，如何调度？
在没有作业长度这种先验知识的情况下，我们如何设计一个调度程序，既能让交互式作业的响应时间最短，又能让周转时间最短？

8.1 MLFQ：基本规则

为了构建这样一个调度程序，本章将介绍多级反馈队列（Multi-level Feedback Queue）背后的基本算法。现有的 MLFQ 实现在细节上各不相同 [E95]，但大多数方法都差不多。

在我们的讨论中，MLFQ 有许多不同的**队列**（queue），每个队列有不同的**优先级**（priority level）。在任何时刻，一个就绪（ready）的作业都只在一个队列中。MLFQ 用优先级来决定某一时刻该运行哪个作业：优先级更高的作业（即位于更高队列中的作业）会被选中运行。

提示：从历史中学习
多级反馈队列（Multi-level Feedback Queue）是一个很好的例子，说明系统可以从过去学习，以此预测未来。这类方法在操作系统中很常见（在计算机科学的许多其他地方也是，包括硬件的分支预测器和缓存（cache）算法）。当作业的行为有阶段性、因而可以预测时，这类方法就能奏效；当然，使用这类技术必须小心，因为它们很容易出错，让系统做出比毫无信息时还糟糕的决策。

当然，同一个队列中可能有不止一个作业，因而它们的优先级相同。这种情况下，我们就在这些作业之间使用时间片轮转（Round-Robin）调度。

于是，我们得到了 MLFQ 的前两条基本规则：

- **规则 1：**如果 A 的优先级 > B 的优先级，运行 A（不运行 B）。
- **规则 2：**如果 A 的优先级 = B 的优先级，A 和 B 按 RR 方式运行。

因此，MLFQ 调度的关键在于调度程序如何设置优先级。MLFQ 并不给每个作业一个固定的优先级，而是根据观察到的行为来改变作业的优先级。例如，如果一个作业在等待键盘输入时反复让出（yield）CPU，MLFQ 就会让它保持高优先级，因为交互式进程大概就是这样的。反之，如果一个作业长时间密集地使用 CPU，MLFQ 就会降低它的优先级。这样，MLFQ 会在进程运行时设法了解它们，从而用作业的历史来预测它未来的行为。

如果把某一时刻各个队列的样子画出来，可能会看到类似下面的情形（第 2 页的图 8.1）。图中，两个作业（A 和 B）处于最高优先级，作业 C 处于中间，作业 D 处于最低优先级。按照我们目前对 MLFQ 工作方式的了解，调度程序只会在 A 和 B 之间轮流分配时间片（time slice），因为它们是系统中优先级最高的作业；可怜的作业 C 和 D 永远都没机会运行，真是岂有此理！

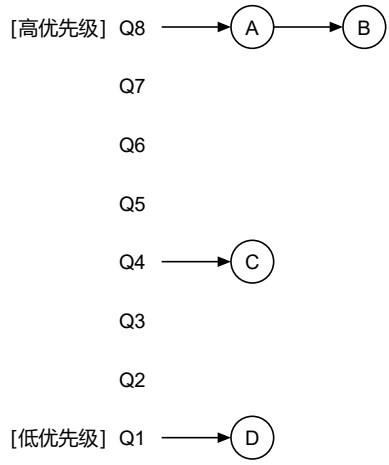


图 8.1：**MLFQ 示例**

 © 2008–23, ARPACI-DUSSEAU

当然，只给出某些队列的一张静态快照（snapshot），并不能真正让你明白 MLFQ 是如何工作的。我们需要了解的是作业的优先级如何随时间变化。而这恰恰就是我们接下来要做的（只有头一回读本书的人才会觉得意外）。

8.2 尝试 1：如何改变优先级

现在我们必须决定，在一个作业的整个生命周期中，MLFQ 如何改变它的优先级，从而改变它所在的队列。这时要记住我们面对的工作负载（workload）：其中既有交互式作业，它们运行时间短，而且可能频繁让出 CPU；也有一些运行时间较长的“CPU 密集型”作业，它们需要大量 CPU 时间，但响应时间并不重要。

为此要引入一个新概念：作业的**配额**（allotment）。配额（allotment）是作业在某个优先级上能用的时间，用完之后调度程序就会降低它的优先级。为简单起见，我们先假设配额（allotment）就等于一个时间片。

下面是我们第一次尝试的优先级调整算法：

- **规则 3：**作业进入系统时，被放在最高优先级，即最上面的队列。
- **规则 4a：**如果作业在运行时用完了它的配额（allotment），它的优先级就会降低（即下移一个队列）。
- **规则 4b：**如果作业在配额（allotment）用完之前就让出了 CPU（例如执行 I/O 操作），它就留在同一优先级，即它的配额被重置。

例 1：单个长时间运行的作业

我们来看几个例子。先看系统中只有一个长时间运行的作业时会发生什么。时间片为 10 毫秒，配额（allotment）设为等于时间片。图 8.2 显示了在有三个队列的调度程序中，这个作业随时间如何变化。

从例子中可以看到，作业以最高优先级（Q2）进入。运行一个 10 毫秒的时间片后，调度程序把它的优先级降低一级，于是作业到了 Q1。在 Q1 运行一个时间片后，作业最终被降到系统中的最低优先级（Q0），并一直留在那里。很简单，对吧？

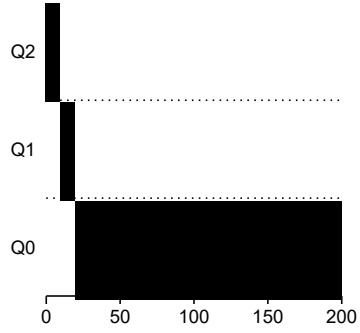


图 8.2：**长时间运行的作业随时间的变化**

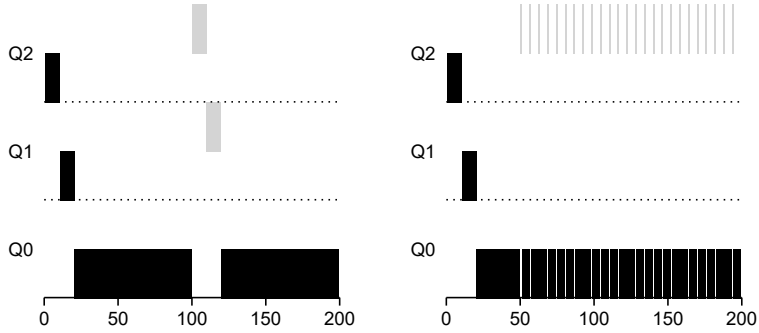


图 8.3：**来了一个交互式作业：两个例子**

例 2：来了一个短作业

现在来看一个更复杂的例子，希望借此看到 MLFQ 如何近似 SJF。这个例子中有两个作业：A 是一个长时间运行的 CPU 密集型作业，B 是一个运行时间很短的交互式作业。假设 A 已经运行了一段时间，然后 B 到达。会发生什么？MLFQ 会对 B 近似 SJF 吗？

第 4 页的图 8.3（左）画出了这个场景的结果。作业 A（黑色）在优先级最低的队列中运行（所有长时间运行的 CPU 密集型作业都会这样）。B（灰色）在 $T = 100$ 时到达，因此被放进最高的队列；它的运行时间很短（只有 20 毫秒），还没降到最底层队列，用两个时间片就运行完了。随后 A 继续以低优先级运行。

从这个例子中，你应该能理解这个算法的一个主要目标：由于算法并不知道一个作业是短是长，它会先假设这个作业很短，给它高优先级。如果确实是短作业，它很快就能运行完；如果不是，它就会慢慢沿着队列下移，很快表明自己是一个长时间运行、更像批处理（batch processing）的进程。MLFQ 就是这样近似 SJF 的。

例 3：I/O 怎么办?

现在来看一个涉及 I/O 的例子。正如上面的规则 4b 所说，如果一个进程在用完配额（allotment）之前就让出了处理器，我们就让它保持在同一优先级。这条规则的用意很简单：例如，如果一个交互式作业在做大量 I/O（比如等待用户的键盘或鼠标输入），它就会在配额（allotment）用完之前让出 CPU；这种情况下，我们不想惩罚这个作业，因此只是让它保持在同一层。

图 8.3（右）展示了这条规则如何起作用：交互式作业 B（灰色）每次只用 1 毫秒 CPU 就去执行 I/O，它和长时间运行的批处理作业 A（黑色）竞争 CPU。由于 B 不断地释放 CPU，MLFQ 让 B 保持在最高优先级；如果 B 是交互式作业，MLFQ 就进一步实现了让交互式作业快速运行的目标。

 © 2008–23, ARPACI-DUSSEAU

提示：调度必须能抵御攻击
你可能觉得调度策略（scheduling policy）和**安全**（security）无关，无论是操作系统内部的调度（如这里讨论的），还是更广泛场景中的调度（例如分布式存储系统处理 I/O 请求时 [Y+18]）；但在越来越多的情况下，调度恰恰是一个安全问题。想想现代数据中心，世界各地的用户在那里共享 CPU、内存、网络和存储系统；如果在策略的设计和执行上不加小心，单个用户就可能损害其他用户，为自己谋利。因此，调度策略是系统安全的重要组成部分，必须精心设计。

当前 MLFQ 的问题

这样，我们就有了一个基本的 MLFQ。它似乎做得相当不错：在长时间运行的作业之间公平地分配 CPU，又让短作业或 I/O 密集型的交互式作业快速运行。遗憾的是，到目前为止我们设计的方法有严重缺陷。你能想到有哪些吗？
（这里请停下来，尽可能狡猾地想一想）

首先，存在**饥饿**（starvation）问题：如果系统中有“太多”交互式作业，它们加起来就会占用全部 CPU 时间，于是长时间运行的作业永远得不到 CPU 时间（它们**饿死**（starve）了）。即使在这种情况下，我们也希望这些作业能有所进展。

其次，聪明的用户可以改写他们的程序，**钻调度程序的空子**（game the scheduler）。钻调度程序的空子（game the scheduler），一般是指要点花招，骗调度程序把超出你应得份额的资源分给你。我们描述的算法容易受到下面这种攻击：在配额（allotment）用完之前，发起一次 I/O 操作（例如访问某个文件），从而让出 CPU；这样你就能留在同一个队列，获得更高比例的 CPU 时间。如果做得好，比如先用掉配额（allotment）的 99% 再让出 CPU，一个作业几乎可以独占 CPU。

最后，一个程序的行为可能随时间改变；原本 CPU 密集的程序可能转入一个交互阶段。按照我们目前的方法，这样的作业就运气不佳了，得不到系统中其他交互式作业那样的待遇。

8.3 尝试 2：优先级提升

我们来试着改变规则，看看能否避免饥饿问题。怎样才能保证 CPU 密集型作业有所进展（哪怕进展不大）呢？

这里的简单想法是周期性地**提升**（boost）系统中所有作业的优先级。实现的办法有很多，我们就做一件简单的事：把它们全都扔进最上面的队列；于是有了一条新规则：

- **规则 5：**经过一段时间 S 后，把系统中所有作业都移到最上面的队列。

这条新规则一下子解决了两个问题。首先，进程一定不会饿死：作业待在最上面的队列中，就会以轮转的方式和其他高优先级作业共享 CPU，因而最终会得到服务。其次，如果一个 CPU 密集型作业变成了交互式的，那么它一旦得到优先级提升（priority boost），调度程序就会正确地对待它。

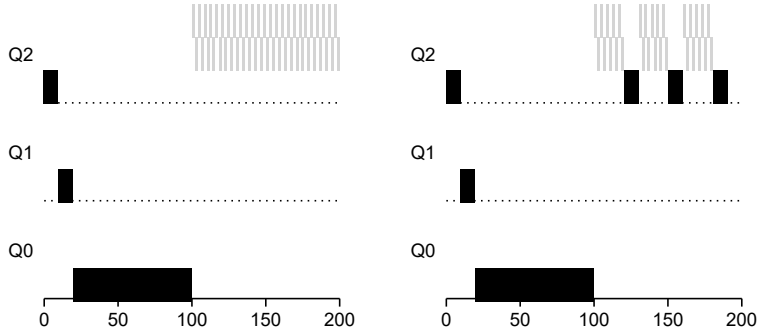


图 8.4：**没有（左）和有（右）优先级提升**

我们来看一个例子。这里只看一个长时间运行的作业在和两个运行时间很短的交互式作业竞争 CPU 时的表现。图 8.4（第 6 页）给出了两张图。左图没有优先级提升（priority boost），因此两个短作业到达后，长时间运行的作业就饿死了；右图每 100 毫秒提升一次优先级（这个值大概太小了，只是为了举例），这样至少保证了长时间运行的作业会有所进展：它每 100 毫秒被提升到最高优先级，从而能周期性地运行。

当然，引入时间段 S 就带来一个显而易见的问题：S 应该设成多少？备受尊敬的系统研究者 John Ousterhout [O11] 过去常把系统中这类值称为 **voo-doo constants**，因为要把它们设对，似乎需要某种黑魔法。遗憾的是，S 就有这种味道。如果设得太高，长时间运行的作业可能会饿死；设得太低，交互式作业又可能得不到合理的 CPU 份额。因此，合适的值往往由系统管理员来找；如今则越来越多地交给基于机器学习的自动方法 [A+17]。

8.4 尝试 3：更好的记账

现在我们还有一个问题要解决：如何防止有人钻我们调度程序的空子？你可能已经猜到，真正的罪魁祸首是规则 4a 和 4b，它们允许作业在配额（allotment）到期之前让出 CPU，从而保住自己的优先级。那么我们该怎么办呢？

这里的解决办法是对 MLFQ 每一层的 CPU 时间做更好的**记账**（accounting）。调度程序不应在进程执行 I/O 时忘掉它在某一层已经用掉了多少配额（allotment），而应记录下来；进程一旦用完配额（allotment），就被降到下一个优先级的队列。它是一次长时间用完配额

提示：避免 voo-doo constants（Ousterhout 定律）

只要有可能，避免 voo-doo constants 都是个好主意。遗憾的是，正如上面的例子所示，这往往很难做到。可以试着让系统自己学习出一个好的值，但这同样不简单。常见的结果是弄出一个写满默认参数值的配置文件，系统运转不太正常时，经验丰富的管理员可以去调整这些值。可以想象，这些值通常没人去改，于是我们只能指望默认值在实际使用中效果不错。这条提示由我们当年的操作系统教授 John Ousterhout 提供，因此我们把它称为 **Ousterhout 定律**（Ousterhout's Law）。

 © 2008–23, ARPACI-DUSSEAU

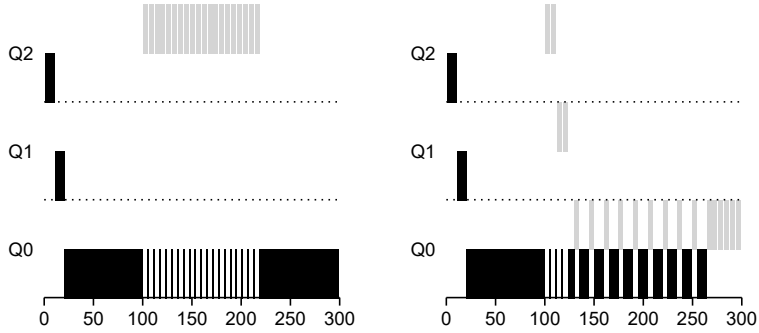


图 8.5：**没有（左）和有（右）防作弊措施**

（allotment），还是分成许多次小段用完，都不应有区别。于是，我们把规则 4a 和 4b 改写成下面这一条规则：

- **规则 4：**一旦作业用完了它在某一层的时间配额（allotment），无论它中间让出了多少次 CPU，它的优先级都会降低（即下移一个队列）。

我们来看一个例子。图 8.5 显示了一个工作负载试图钻调度程序空子时的情况：左边用的是旧的规则 4a 和 4b，右边用的是新的防作弊规则 4。如果没有任何防作弊保护，进程就可以在配额（allotment）用完之前发起一次 I/O，从而留在同一优先级，独占 CPU 时间。右图有了更好的记账（accounting），无论进程的 I/O 行为如何，它都会慢慢地沿着队列下移，因而无法获得不公平的 CPU 份额。

8.5 MLFQ 的调优及其他问题

MLFQ 调度还会引出其他一些问题。一个大问题是这样的调度程序该如何**参数化**（parameterize）。例如，应该有多少个队列？每个队列的时间片应该多大？配额（allotment）呢？为了避免饥饿并顾及行为的变化，应该多久提升一次优先级？这些问题都没有简单的答案，只有对工作负载积累一些经验，再对调度程序做相应的调优，才能达到令人满意的平衡。

例如，大多数 MLFQ 变体允许不同队列使用不同长度的时间片。高优先级的队列通常分到较短的时间片；毕竟这些队列里是交互式作业，在它们之间快速切换（例如 10 毫秒或更短）是合理的。相比之下，低优先级的队列里是长时间运行的 CPU 密集型作业，因此较长的时间片（例如几百毫秒）效果很好。图 8.6 展示了一个例子：两个作业在最高的队列中运行 20 毫秒（时间片为 10 毫秒），在中间的队列中运行 40 毫秒（时间片为 20 毫秒），在最低的队列中时间片为 40 毫秒。

Solaris 的 MLFQ 实现，即分时（Time-Sharing，TS）调度类，配置起来特别方便。它提供了一组表，精确规定进程的优先级在整个生命周期中如何变化、每个时间片有多长、多久提升一次作业的优先级 [AD00]；管理员可以随意摆弄这些表，让调度程序表现出不同的行为。这些表的默认配置是 60 个队列，时间片长度从 20 毫秒（最高优先级）缓慢增加到几百毫秒（最低优先级），优先级大约每 1 秒提升一次。

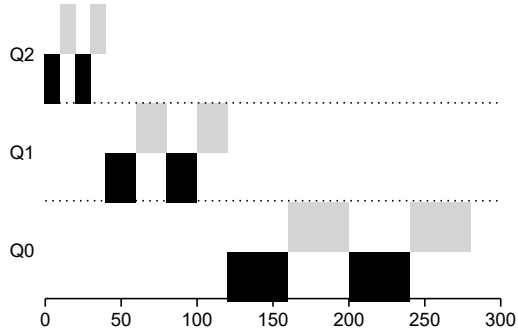


图 8.6：**优先级越低，时间片越长**

其他 MLFQ 调度程序不使用表，也不使用本章描述的这些具体规则，而是用数学公式来调整优先级。例如，FreeBSD 4.3 版的调度程序根据进程用掉了多少 CPU，用一个公式计算作业当前的优先级 [LM+89]；此外，CPU 使用量会随时间衰减，从而用一种不同于本章的办法实现所需的优先级提升（priority boost）。Epema 的论文对这类 **decay-usage** 算法及其性质做了很好的综述 [E95]。

最后，许多调度程序还有其他一些特性，你也可能碰到。例如，有些调度程序把最高的几个优先级留给操作系统的工作；因此，普通的用户作业永远无法获得系统中最高的那几级优先级。有些系统还允许用户提供一些**建议**（advice）来帮助设置优先级；例如，使用命令行工具 `nice`，就可以（在一定程度上）提高或降低作业的优先级，从而增加或减少它在任一时刻运行的机会。详情请见手册页（manual page）。

8.6 MLFQ：小结

我们介绍了一种叫作多级反馈队列（MLFQ）的调度方法。希望你现在能明白它为什么叫这个名字：它有多级队列，并利用反馈来确定某个作业的优先级。历史是它的向导：留意作业随时间的行为，并据此对待它们。

本章各处陆续改进的 MLFQ 规则，在这里汇总一下，方便查看：

- **规则 1：** 如果 A 的优先级 > B 的优先级，运行 A（不运行 B）。
- **规则 2：** 如果 A 的优先级 = B 的优先级，A 和 B 用所在队列的时间片以轮转方式运行。
- **规则 3：** 作业进入系统时，被放在最高优先级，即最上面的队列。

提示：尽量利用建议

操作系统很少清楚什么对系统中的每个进程最好，因此提供一些接口，让用户或管理员给操作系统一些**提示**（hint），往往很有用。我们常把这类提示称为**建议**（advice），因为操作系统不必照办，只是可以把它纳入考虑，以便做出更好的决策。这类提示在操作系统的许多部分都很有用，包括调度程序（例如 `nice`）、内存管理器（例如 `madvise`），以及文件系统（file system，例如 informed prefetching and caching [P+95]）。

- **规则 4：**一旦作业用完了它在某一层的时间配额（allotment），无论它中间让出了多少次 CPU，它的优先级都会降低（即下移一个队列）。
- **规则 5：**经过一段时间 S 后，把系统中所有作业都移到最上面的队列。

MLFQ 有意思的地方在于：它不要求预先知道作业的性质，而是观察作业的执行情况，据此确定它的优先级。这样，它做到了两全其美：对运行时间很短的交互式作业，它能提供出色的整体性能（和 SJF/STCF 相近）；对长时间运行的 CPU 密集型工作负载，它又是公平的，能让它们有所进展。正因如此，许多系统，包括 BSD UNIX 的衍生版本 [LM+89, B86]、Solaris [M06]，以及 Windows NT 和之后的 Windows 操作系统 [CS97]，都用某种形式的 MLFQ 作为基础调度程序。

参考文献

[A+17] "Automatic Database Management System Tuning Through Large-scale Machine Learning" by Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, Bohan Zhang. SIGMOD '17. *This isn't about the application of machine learning to CPU scheduling in the OS, but rather a cool early example of automatically tuning parameters of a database via ML techniques. Worth a read, if you like ML… which, alas, everyone seems to these days.*

[AD00] "Multilevel Feedback Queue Scheduling in Solaris" by Andrea Arpaci-Dusseau. Available: http://www.ostep.org/Citations/notes-solaris.pdf. *A great short set of notes by one of the authors on the details of the Solaris scheduler. OK, we are probably biased in this description, but the notes are pretty darn good.*

[B86] "The Design of the UNIX Operating System" by M.J. Bach. Prentice-Hall, 1986. *One of the classic old books on how a real UNIX operating system is built; a definite must-read for kernel hackers.*

[C+62] "An Experimental Time-Sharing System" by F. J. Corbato, M. M. Daggett, R. C. Daley. IFIPS 1962. *A bit hard to read, but the source of many of the first ideas in multi-level feedback scheduling. Much of this later went into Multics, which one could argue was the most influential operating system of all time.*

[CS97] "Inside Windows NT" by Helen Custer and David A. Solomon. Microsoft Press, 1997. *The NT book, if you want to learn about something other than UNIX. Of course, why would you? OK, we're kidding; you might actually work for Microsoft some day you know.*

[E95] "An Analysis of Decay-Usage Scheduling in Multiprocessors" by D.H.J. Epema. SIGMETRICS '95. *A nice paper on the state of the art of scheduling back in the mid 1990s, including a good overview of the basic approach behind decay-usage schedulers.*

[LM+89] "The Design and Implementation of the 4.3BSD UNIX Operating System" by S.J. Leffler, M.K. McKusick, M.J. Karels, J.S. Quarterman. Addison-Wesley, 1989. *Another OS classic, written by four of the main people behind BSD. The later versions of this book, while more up to date, don't quite match the beauty of this one.*

[M06] "Solaris Internals: Solaris 10 and OpenSolaris Kernel Architecture" by Richard McDougall. Prentice-Hall, 2006. *A good book about Solaris and how it works.*

[O11] "John Ousterhout's Home Page" by John Ousterhout. www.stanford.edu/~ouster/. *The home page of the famous Professor Ousterhout. The two co-authors of this book had the pleasure of taking graduate operating systems from Ousterhout while in graduate school; indeed, this is where the two co-authors got to know each other, eventually leading to marriage, kids, and even this book. Thus, you really can blame Ousterhout for this entire mess you're in.*

[P+95] "Informed Prefetching and Caching" by R.H. Patterson, G.A. Gibson, E. Ginting, D. Stodolsky, J. Zelenka. SOSP '95, Copper Mountain, Colorado, October 1995. *A fun paper about some very cool ideas in file systems, including how applications can give the OS advice about what files it is accessing and how it plans to access them.*

[Y+18] "Principled Schedulability Analysis for Distributed Storage Systems using Thread Architecture Models" by Suli Yang, Jing Liu, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. OSDI '18, San Diego, California. *A recent work of our group that demonstrates the difficulty of scheduling I/O requests within modern distributed storage systems such as Hive/HDFS, Cassandra, MongoDB, and Riak. Without care, a single user might be able to monopolize system resources.*

 © 2008–23, ARPACI-DUSSEAU

作业（模拟）

程序 `mlfq.py` 可以让你看到本章介绍的 MLFQ 调度程序是如何工作的。详情请见 README。

问题

1. 随机生成几个只有两个作业、两个队列的问题来运行；计算每个问题的 MLFQ 执行轨迹。限制每个作业的长度并关闭 I/O，会让你轻松一些。
2. 要重现本章中的每个例子，应该怎样运行调度程序?
3. 应该怎样配置调度程序的参数，才能让它的行为和时间片轮转（Round-Robin）调度程序完全一样?
4. 构造一个包含两个作业的工作负载和一组调度程序参数，让其中一个作业利用旧的规则 4a 和 4b（用 `-S` 选项打开）钻调度程序的空子（game the scheduler），在某段时间内获得 99% 的 CPU。
5. 假设一个系统最高队列的量子长度（quantum length）为 10 毫秒，为了保证一个长时间运行（且可能饿死）的作业至少获得 5% 的 CPU，你需要多久把作业提升回最高优先级一次（用 `-B` 选项)?
6. 调度中有一个问题：刚完成 I/O 的作业应该加到队列的哪一端；`-I` 选项可以改变调度模拟器在这方面的行为。试一些工作负载，看看能否观察到这个选项的效果。